



Università
di Catania

DIPARTIMENTO DI MATEMATICA E INFORMATICA
CORSO DI LAUREA MAGISTRALE IN INFORMATICA

Alberto Bocchieri

Relazione progetto Sistemi Cloud

Stima della disponibilità di parcheggio su segmenti stradali reali
e deploy su AWS

*Docenti: Giuseppe Pappalardo e
Salvatore Nicotra*

Anno Accademico 2025–2026

Indice

1	Introduzione ed esperienza utente	1
1.1	Esperienza utente e GUI	2
1.1.1	Mappa, interazione e rappresentazione dei segmenti	3
1.1.2	Come leggere percentuali e colori	4
2	Architettura applicativa e dati geografici	6
2.1	Responsabilità dei servizi	7
2.2	Dati geografici, OSM e modello dati	8
2.2.1	Importazione OSM e tipi di sosta	9
3	Predizione, TomTom e suggerimenti AI	11
3.1	Calcolo della predizione	11
3.2	TomTom con cache e controllo del budget	13
3.3	Percorso del dato e suggerimenti AI	13
3.3.1	Nemotron e Text-to-Speech	14
4	Esecuzione cloud, Infrastructure as Code e CI/CD	16
4.1	Docker Compose e simulazione k3d	17
4.2	Deploy reale su AWS	18
4.3	Infrastructure as Code, CI/CD, sicurezza e test	19
4.3.1	GitHub Actions, ECR e gestione dei segreti	19
4.3.2	Controlli automatici e verifica operativa	20
5	Limiti, sviluppi possibili e conclusioni	22
5.1	Sviluppi possibili	23
5.2	Conclusioni	23
	Bibliografia	24

Capitolo 1

Introduzione ed esperienza utente

Trovare parcheggio in città sembra un problema semplice finché non si prova a trasformarlo in un dato. Le informazioni disponibili raramente dicono quanti posti siano liberi lungo una strada in un preciso momento. Il traffico descrive quanto una via sia congestionata, OpenStreetMap descrive la rete stradale e alcuni tipi di sosta, mentre le segnalazioni degli utenti raccontano solo episodi puntuali. ParcheggIA nasce dall'idea di mettere insieme questi segnali e restituire un'indicazione comprensibile durante la guida.

Il progetto non divide Catania in quartieri colorati in modo uniforme. L'unità principale è un piccolo segmento di strada, in genere delimitato da un incrocio o da una lunghezza massima di circa 120 metri. Ogni segmento ha un nome, una geometria, un tipo di sosta e una percentuale stimata. In questo modo due vie vicine possono ricevere valutazioni differenti, cosa che sarebbe impossibile usando una sola percentuale per un'intera zona.

La percentuale non rappresenta il numero esatto di stalli liberi. Va letta come una stima pratica della probabilità di trovare posto. Questa distinzione è importante perché le API impiegate nel progetto forniscono traffico, incidenti e punti di interesse, ma non l'occupazione in tempo reale di ogni parcheggio su strada. Il sistema costruisce quindi una previsione propria, dichiarandone anche il livello di affidabilità.

Fin dall'inizio sono stati considerati due obiettivi. Il primo era ottenere una demo piacevole da usare, vicina all'aspetto di un navigatore. Il secondo era mostrare una vera architettura distribuita: servizi separati, dati geografici, cache, eventi, container, Kubernetes e una versione su AWS. La stessa applicazione può essere avviata con Docker Compose, eseguita in un cluster k3d locale oppure distribuita su EKS con database e broker gestiti.

Un altro requisito pratico era la replicabilità. Chi clona la repository deve poter vedere segmenti reali, heatmap, suggerimenti e sintesi vocale anche senza possedere chiavi TomTom, NVIDIA o ElevenLabs. Per questo le integrazioni esterne sono opzionali e hanno modalità simulate che mantengono attivo il percorso completo dell'applicazione.

Il codice sorgente, gli script di avvio e i manifest dell'infrastruttura sono disponibili nella repository GitHub del progetto: <https://github.com/AlbertoBocchieri/Progetto-Sistemi-Cloud>.

La Figura 1.1 riassume le famiglie di tecnologie usate. Si tratta di una vista concettuale: nei capitoli successivi i collegamenti vengono descritti con maggiore precisione.

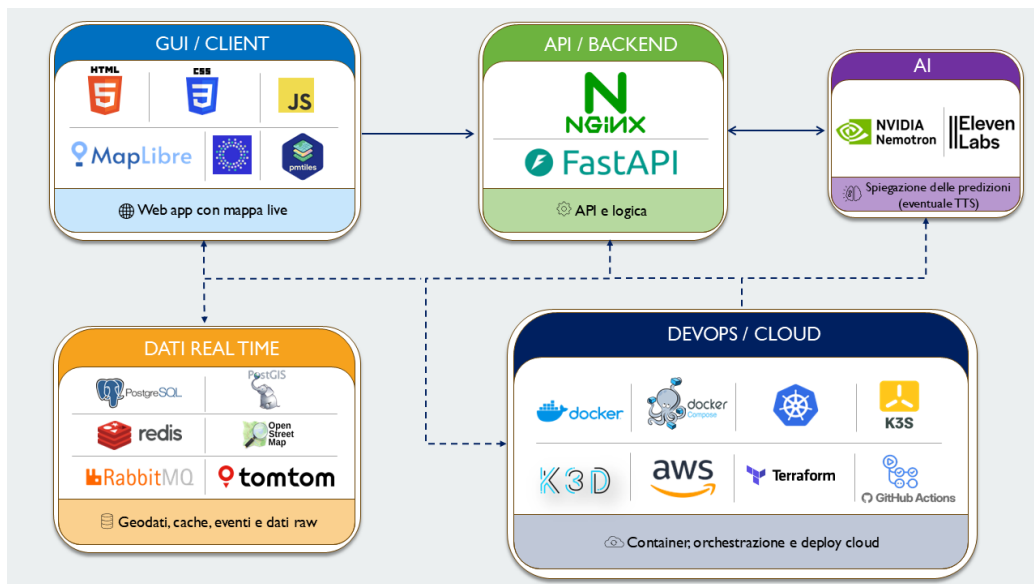


Figura 1.1: Panoramica delle tecnologie usate nel progetto.

1.1 Esperienza utente e GUI

La GUI è stata progettata come un'interfaccia di guida. La mappa occupa quasi tutto lo schermo; i comandi hanno dimensioni grandi e le informazioni più importanti restano vicine alla posizione dell'utente. Durante lo sviluppo sono stati eliminati pannelli che duplicavano dati già visibili sulla mappa, perché in un contesto di guida ogni elemento aggiunto riduce lo spazio disponibile e aumenta la distrazione.

La schermata principale, mostrata in Figura 1.2, contiene la posizione orientata, la heatmap, i marker percentuali e un suggerimento operativo. La barra inferiore permette di aprire parcheggi, preferiti e impostazioni. Il tema può essere chiaro o scuro e il pulsante del volume controlla la lettura vocale dei consigli.

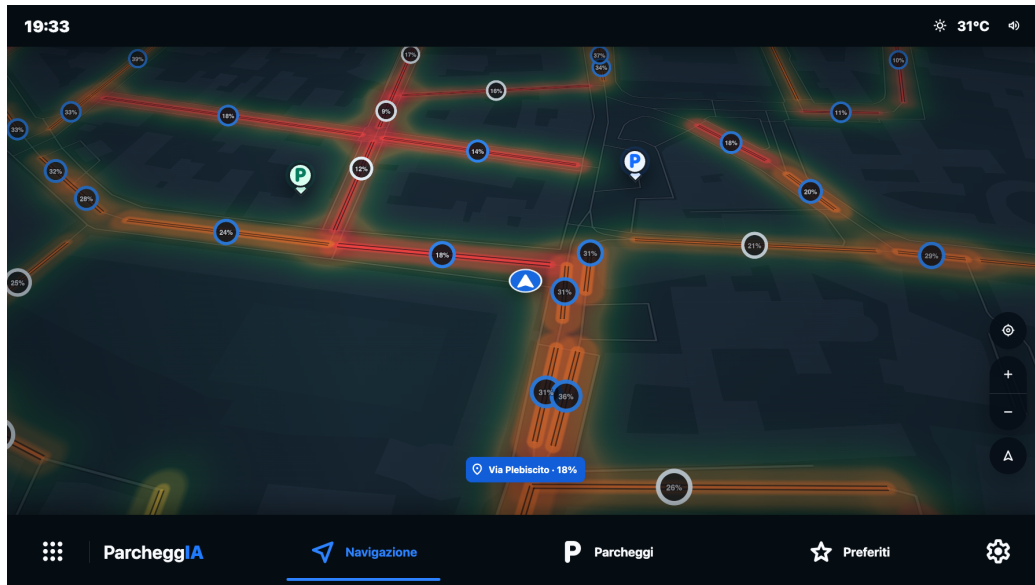


Figura 1.2: Interfaccia principale di ParcheggIA durante la simulazione di guida.

1.1.1 Mappa, interazione e rappresentazione dei segmenti

La mappa viene disegnata nel browser con MapLibre GL JS [1]. Rispetto a una libreria orientata a mappe bidimensionali tradizionali, MapLibre permette di usare WebGL, inclinare la camera, modificare l'orientamento in base al senso di marcia e sovrapporre numerosi layer vettoriali senza ricreare l'intera pagina. Queste caratteristiche sono state necessarie per ottenere una visuale simile a quella di un navigatore.

La cartografia di base di Catania è contenuta nel file locale `frontend/assets/catania.pmtiles`. PMTiles raccoglie le tile vettoriali in un singolo archivio consultabile a intervalli [2], quindi il browser legge solo le parti necessarie alla visuale corrente. La mappa non richiede una chiave MapTiler o TomTom Map Display e continua a funzionare anche senza connessione

verso un fornitore cartografico. La copertura dettagliata resta però limitata all'area preparata per il progetto.

La heatmap segue le geometrie stradali. Ogni linea viene campionata in più punti e combinata con due nastri lineari, uno più largo e sfumato e uno centrale più definito. Il risultato evita l'effetto di una serie di cerchi isolati. Il colore indica la difficoltà stimata, mentre la trasparenza e la morbidezza dipendono dall'affidabilità della previsione.

I marker dei segmenti mostrano la percentuale al centro e usano il bordo per distinguere il tipo di sosta. I parcheggi individuati tramite TomTom Search, oppure simulati in assenza della chiave, hanno marker separati. Quando entrano nel campo visivo compaiono con una breve rotazione; quando vengono consigliati dal servizio AI ricevono un anello verde animato. Questi effetti hanno una funzione precisa: attirare l'attenzione su un punto appena diventato rilevante senza lasciare animazioni permanenti su tutta la mappa.

Da PC ci si può spostare cliccando su una strada. Il punto scelto viene agganciato alla rete OSM locale e diventa la nuova posizione. La demo include anche un pilota automatico di circa 500 metri, usato per verificare cambio del segmento, orientamento della freccia, comparsa dei marker e aggiornamento dei suggerimenti senza dover guidare realmente.

1.1.2 Come leggere percentuali e colori

La GUI traduce la percentuale in cinque stati, riportati nella Tabella 1.1. Le soglie sono volutamente semplici: servono a comunicare rapidamente il risultato e non devono essere confuse con una certezza statistica. La confidence, cioè l'affidabilità della fonte, viene usata soprattutto per modulare l'aspetto della heatmap e per costruire il contesto del suggerimento.

Tabella 1.1: Soglie usate per interpretare la percentuale.

Intervallo	Significato	Colore prevalente
< 20%	molto difficile	rosso
20–39%	difficile	rosso/arancio
40–59%	incerto	giallo
60–79%	buono	verde
≥ 80%	favorevole	verde intenso

Il chip blu vicino alla posizione mostra soltanto il nome della strada corrente e la percentuale. Le informazioni più dettagliate, come tariffa, tipo di sosta, fonte e affidabilità, rimangono disponibili nel pannello del segmento. Questa separazione mantiene pulita la vista principale ma non nasconde i limiti del dato a chi vuole approfondire.

Capitolo 2

Architettura applicativa e dati geografici

L'applicazione è divisa in servizi con responsabilità distinte. La separazione non è stata introdotta perché il carico della demo richiedesse necessariamente molti processi: un unico backend sarebbe stato sufficiente dal punto di vista prestazionale. La scelta serve a rendere visibili i confini tipici di un sistema cloud, a poter sostituire i componenti locali con servizi gestiti e a isolare le integrazioni più lente o facoltative.

Il browser raggiunge prima il container frontend, basato su Nginx. I file HTML, CSS e JavaScript vengono serviti direttamente, mentre le richieste con prefisso `/api/v1/`, `/ingestion/` e `/ai/` sono inoltrate all'API Gateway. Anche il gateway è Nginx: non contiene logica applicativa, ma smista ogni percorso verso il servizio FastAPI corretto. In questo modo il browser usa un solo punto di ingresso e non conosce gli indirizzi interni dei container.

La Figura 2.1 mostra i livelli principali senza disegnare ogni singola chiamata. Le frecce continue rappresentano il percorso HTTP e l'accesso ai dati; quelle tratteggiate indicano integrazioni opzionali.

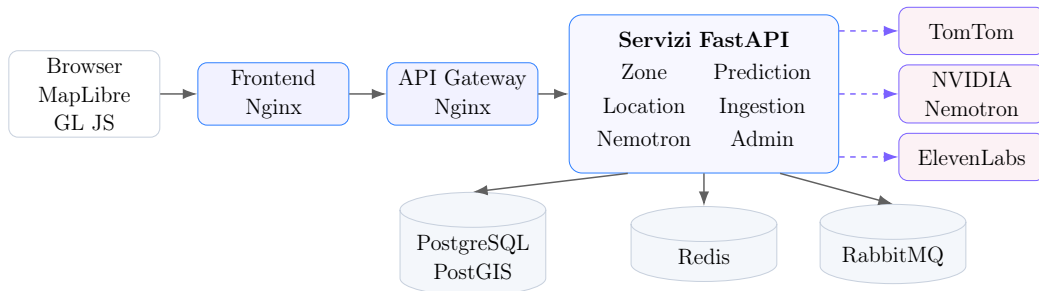


Figura 2.1: Architettura logica e punti di ingresso dell'applicazione.

2.1 Responsabilità dei servizi

La Tabella 2.1 riporta le porte pubblicate da Docker Compose. All'interno della rete Docker ogni servizio FastAPI ascolta sulla porta 8000; la mappatura esterna serve soprattutto per sviluppo e diagnosi. Nell'uso normale il frontend passa comunque attraverso il gateway sulla porta 8000 e non contatta direttamente le altre porte.

Tabella 2.1: Servizi applicativi, porte locali e dipendenze principali.

Servizio	Ruolo	Dipendenze principali
frontend (8080)	Serve la GUI, la basemap PMTiles e inoltra le richieste applicative.	API Gateway
api-gateway (8000)	Smista i percorsi HTTP verso i servizi interni.	Tutti i servizi FastAPI
zone-service (8001)	Gestisce segmenti, rete stradale, query spaziali, report e consumer degli eventi.	PostGIS, Redis, RabbitMQ
ingestion-service (8002)	Legge scenari e dati TomTom, applica cache e limiti di consumo, quindi pubblica eventi.	Redis, RabbitMQ, TomTom opzionale
nemotron-service (8003)	Genera suggerimenti e audio, oppure usa la modalità simulata.	NVIDIA ed ElevenLabs opzionali
prediction-service (8004)	Calcola percentuali, confidence, tempi di ricerca e heatmap.	PostGIS, Redis
location-service (8005)	Mantiene la sessione live e orchestra segmento corrente, vicini e predizione.	Zone, Prediction, Redis, RabbitMQ
admin-service (8006)	Espone diagnostica, scenari demo e operazioni di reset.	Zone, Redis, RabbitMQ

Il `location-service` è il punto centrale del flusso legato alla posizione. Salva la sessione in Redis, interroga il `zone-service` per trovare il segmento corrente e i tratti entro 500 metri, poi chiede al `prediction-service` la percentuale associata. Infine pubblica su RabbitMQ un evento `user.location.updated`. Il suggerimento AI segue invece una richiesta separata della GUI e non passa dal servizio di localizzazione.

Redis e RabbitMQ non sono intercambiabili. Redis conserva informazioni che devono essere lette velocemente, come sessioni, cache delle predizioni, segnali temporanei e contatori di budget. RabbitMQ trasporta eventi che possono essere elaborati in modo asincrono. Se il servizio di ingestion riceve nuovi segnali di traffico, può pubblicarli senza attendere che tutti i segmenti siano aggiornati nella stessa richiesta HTTP.

2.2 Dati geografici, OSM e modello dati

La parte geografica usa due rappresentazioni della città che hanno scopi diversi. L'archivio PMTiles viene letto dal browser e serve a disegnare strade, edifici e nomi sulla mappa. PostgreSQL/PostGIS contiene invece geometrie interrogabili dal backend. Una tile vettoriale è adatta alla visualizzazione, ma non è una base affidabile per collegare report, tipi di sosta e stime persistenti. Per questo i segmenti sono importati separatamente da OpenStreetMap, seguendo anche lo schema dedicato alla sosta stradale [3].

Il dato principale è la tabella `parking_segments`. Ogni riga contiene identificativo, nome della via, geometria lineare, lunghezza, tipo di sosta, zona tariffaria, fonte e confidence della fonte. Le tabelle `road_nodes` e `road_edges` descrivono una rete navigabile distinta: viene usata per agganciare il movimento demo alla strada senza dipendere dalla continuità dei soli segmenti di parcheggio.

I parcheggi fuori strada sono rappresentati da `parking_lots`, mentre `segment_reports` conserva le segnalazioni degli utenti. Gli eventi e i segnali più brevi non vengono duplicati nel database: rimangono in Redis sotto chiavi come `segment:signals:*`. La Figura 2.2 mette in evidenza questa separazione.

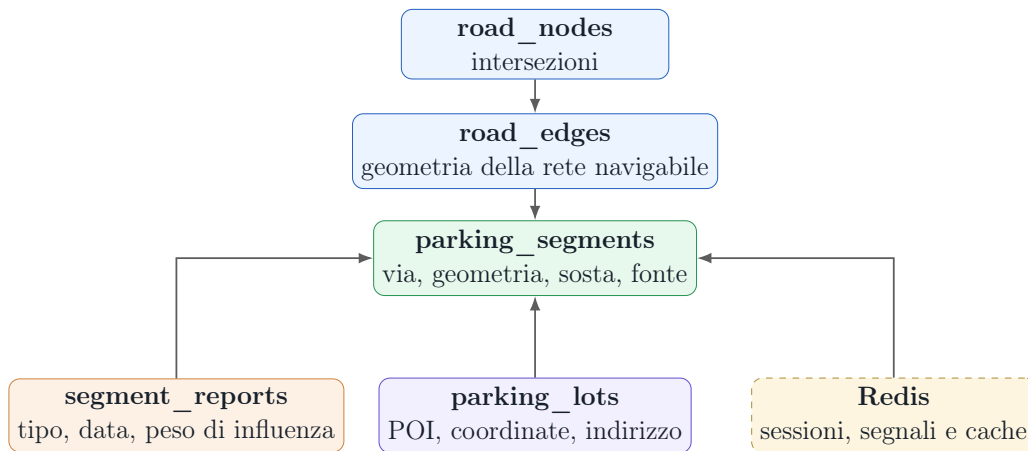


Figura 2.2: Entità persistenti e stato temporaneo usati per i segmenti.

2.2.1 Importazione OSM e tipi di sosta

La pipeline offline parte dalla rete stradale di Catania. Le geometrie vengono pulite, divise in corrispondenza degli incroci e ulteriormente spezzate quando superano la lunghezza prevista. Lo script genera SQL riproducibile, conservato in `data/osm/catania_segments.sql`, che viene caricato dal job `db-init` al primo avvio. Questo permette a Docker Compose e Kubernetes di costruire lo stesso insieme di segmenti senza dipendere da una chiamata online durante la demo.

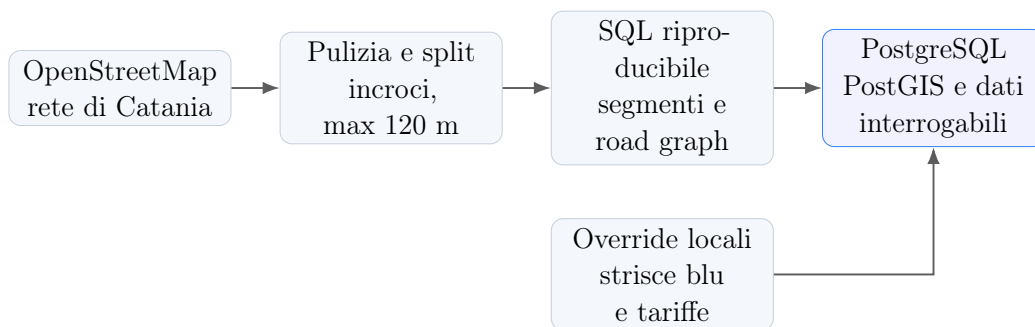


Figura 2.3: Preparazione offline della rete stradale e degli override.

Le informazioni sulla sosta disponibili in OSM non sono sempre complete. Dopo l'importazione vengono quindi applicati override locali, preparati a partire da fonti AMTS, documenti comunali e controlli manuali. Un segmento può essere classificato come `blue`, `probable_free`, `restricted` oppure `unknown`. “Probabile libero” non significa che la sosta sia garantita o

legalmente consentita: indica soltanto che non risultano regole tariffarie o limitazioni note nelle fonti disponibili.

Il controllo contro la cartografia ha richiesto particolare attenzione. Durante le prime importazioni alcuni tratti ereditavano nomi o geometrie che non coincidevano con la strada visibile. La soluzione adottata è stata privilegiare i segmenti con identificativo `ct-osm-*`, collegati alla rete importata, e mantenere i vecchi segmenti dimostrativi soltanto come compatibilità quando il database OSM non è ancora inizializzato. I controlli automatici verificano inoltre che le geometrie siano valide e appoggiate a una strada reale.

Capitolo 3

Predizione, TomTom e suggerimenti AI

3.1 Calcolo della predizione

Il modello attuale è basato su regole, non su un algoritmo addestrato con uno storico di occupazione. La scelta permette di osservare ogni contributo al risultato e di usare il sistema anche prima di raccogliere dati sufficienti per un modello statistico. La percentuale finale viene calcolata con la formula:

$$P = \text{clamp}_{[5,95]}(B_{\text{tipo}} + \Delta_{\text{report}} + \Delta_{\text{segnali}} + \Delta_{\text{tempo}}) \quad (3.1)$$

B_{tipo} è il valore iniziale legato al tipo di sosta. Per i segmenti OSM senza segnali reali non viene usato sempre lo stesso numero: una funzione deterministica basata su via, tipo e identificativo distribuisce la stima dentro un intervallo plausibile. A parità di dati il risultato rimane stabile tra due avvii, quindi la mappa non cambia casualmente a ogni aggiornamento.

Tabella 3.1: Baseline e intervalli della simulazione senza dati esterni.

Tipo di sosta	Baseline	Intervallo simulato	Confidence iniziale
Strisce blu	42%	28–72%	70%
Probabile libero	48%	22–78%	52%
Limitato	14%	5–24%	66%
Sconosciuto	42%	24–62%	42%

Le segnalazioni degli ultimi trenta minuti producono una correzione limitata tra -15 e $+15$. Un posto trovato vale $+5$, un posto appena liberato $+6$, una zona piena -7 e una chiusura -8 , moltiplicati per il peso di influenza del report. La forma usata nel servizio è:

$$\Delta_{report} = \text{clamp}_{[-15,15]}(5F + 6L - 7Z - 8C) \quad (3.2)$$

dove F , L , Z e C rappresentano rispettivamente segnalazioni di posto trovato, posto liberato, zona piena e parcheggio chiuso. I report influenzano anche i segmenti entro 150 metri con peso decrescente, così una singola segnalazione non modifica in modo identico un'intera area.

I segnali presenti in Redis combinano disponibilità di parcheggi, pressione del traffico e pressione degli eventi. La correzione è limitata tra -35 e $+20$:

$$\Delta_{segnali} = \text{clamp}_{[-35,20]}(35(A - 0.5) - 22T - 18E) \quad (3.3)$$

A è un eventuale indicatore di disponibilità tra 0 e 1, T la pressione del traffico ed E quella degli incidenti. Se nessun segnale è presente, la correzione vale zero. Nel fine settimana viene aggiunto $+3$; tra le 8 e le 10 oppure tra le 17 e le 20 si applica -7 ; di notte, tra mezzanotte e le 6, si applica $+6$. La distanza non entra direttamente nell'Equazione 3.1: serve a scegliere i segmenti vicini e a ordinare il contesto mostrato all'utente.

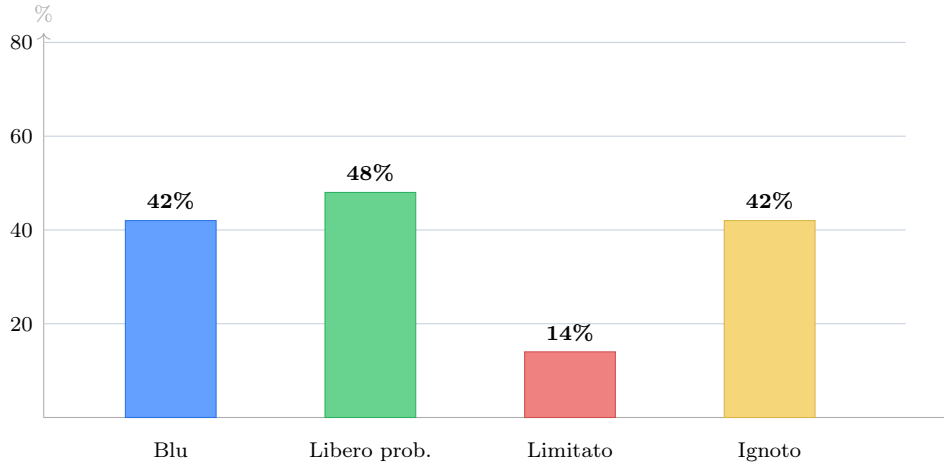


Figura 3.1: Baseline prima dell'applicazione delle correzioni dinamiche.

Oltre alla percentuale viene calcolata la confidence. La base dipende dal tipo di sosta e cresce in parte con l'affidabilità della fonte del segmento.

Segnali o report abbastanza forti aumentano ulteriormente la confidence, fino a un massimo del 92%. Questa misura non cambia il colore semantico del risultato: rende più o meno definita la visualizzazione e aiuta il servizio AI a formulare consigli prudenti.

3.2 TomTom con cache e controllo del budget

TomTom viene interrogato esclusivamente dal backend attraverso le API Traffic e Search [4, 5]. Il frontend invia al servizio di ingestion la posizione corrente e un raggio di 500 metri; il servizio decide se esiste già una risposta valida per la cella geografica. Le celle sono larghe circa 250 metri e la cache del traffico dura almeno cinque minuti. Muoversi più volte nella stessa area non produce quindi una richiesta esterna per ogni aggiornamento della posizione.

Traffic Flow restituisce velocità corrente, velocità a traffico libero, tempo di percorrenza, confidence e possibili chiusure. Traffic Incidents restituisce incidenti, lavori, code, ritardi e gravità dell'evento in una bounding box. Una stima live può usare fino a cinque campioni Flow e una richiesta Incidents per coprire l'area; i risultati vengono poi propagati ai segmenti vicini con peso spaziale. Non viene mai effettuata una chiamata TomTom per ogni segmento.

Search API ha uno scopo diverso: individua aree di parcheggio, ma non conosce quanti posti siano liberi. Questi POI vengono mantenuti in cache molto più a lungo perché nome e posizione cambiano raramente. La GUI li mostra come marker e può usarli nel contesto inviato al servizio AI.

Redis conserva anche contatori mensili per servizio. Nella configurazione di test il limite locale predefinito è il 5% della quota disponibile; superata questa soglia, la chiamata viene rifiutata in modo controllato. In assenza della chiave TomTom, gli stessi campi vengono prodotti dalla simulazione deterministica e la GUI continua a funzionare senza mostrare errori bloccanti.

3.3 Percorso del dato e suggerimenti AI

Il modo più semplice per capire l'architettura è seguire una posizione dall'interazione dell'utente fino alla mappa. Quando viene premuto il pilota automatico, usato il GPS oppure cliccata una strada, il frontend aggiorna subito il marker per mantenere fluida l'animazione. Gli aggiornamenti più costosi

vengono invece ritardati e raggruppati, evitando di ridisegnare heatmap e dati remoti a ogni fotogramma.

La prima riga della Figura 3.2 riassume il percorso della posizione. Il `location-service` ottiene dal `zone-service` il segmento corrente e i tratti vicini, mentre il `prediction-service` legge baseline, report e segnali. La risposta torna attraverso il gateway e viene trasformata dal frontend in sorgenti GeoJSON per MapLibre.

La seconda riga mostra il percorso separato del suggerimento. Quando cambia il segmento o cambia in modo significativo la percentuale, la GUI prepara un contesto con le vie e i parcheggi realmente presenti nelle vicinanze. La richiesta passa dal gateway al `nemotron-service`, che può chiamare NVIDIA e successivamente ElevenLabs. Le risposte percorrono a ritroso lo stesso canale HTTP; le frecce del diagramma mostrano il verso della richiesta per evitare linee sovrapposte.

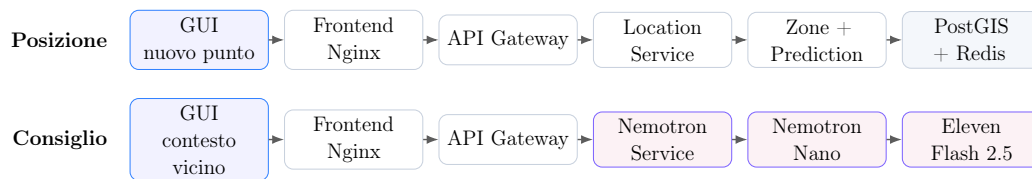


Figura 3.2: I due flussi principali: aggiornamento della posizione e suggerimento vocale.

3.3.1 Nemotron e Text-to-Speech

Il modello configurato è `nvidia/nemotron-3-nano-30b-a3b` [6]. La variante Nano è stata scelta dopo aver provato un modello più grande: per un consiglio breve il guadagno di velocità era più utile della maggiore capacità del modello Super. Il prompt riceve nome e percentuale del segmento corrente, vie vicine, distanza, tipo di sosta, confidence e parcheggi disponibili. Gli identificativi dei target vengono inclusi nel contesto e la risposta viene validata, così il modello non può evidenziare una via o un parcheggio non presenti nei dati ricevuti.

Il testo richiesto è pratico e in italiano. Non deve esporre formule, parlare di “stima inferita” o aggiungere avvertenze generiche che occupano tempo durante la guida. La card mostra la parte essenziale in non più di due righe; il testo vocale può aggiungere una breve motivazione, ma rimane sintetico. Le risposte vengono memorizzate in cache usando modello, segmento, percen-

tuale, stato e contesto, evitando di consumare una nuova richiesta quando la situazione non è cambiata.

La Figura 3.3 mostra il dettaglio spostato dalla sezione iniziale: qui è più utile perché permette di osservare insieme il suggerimento, il segmento corrente e i marker circostanti.



Figura 3.3: Dettaglio del suggerimento durante la guida simulata.

Per la sintesi vocale viene usato Eleven Flash 2.5 tramite l'API Eleven-Labs [7], con similarity impostata al 50% e style al 25%. Il modello Flash consuma meno crediti e riduce la latenza rispetto a modelli più pesanti. Il servizio restituisce audio MP3; il browser lo riproduce e chiude la card con un'animazione quando il parlato termina. Il tasto volume annulla immediatamente la riproduzione e impedisce nuove letture finché non viene riattivato.

Senza chiave NVIDIA il servizio produce un suggerimento simulato usando soltanto i dati ricevuti. Senza ElevenLabs, il frontend usa la Web Speech API del browser. Questi fallback non cercano di imitare perfettamente i fornitori esterni: mantengono disponibile la stessa interazione e rendono possibile valutare l'architettura dopo un semplice `docker compose up`.

Capitolo 4

Esecuzione cloud, Infrastructure as Code e CI/CD

Lo stesso codice applicativo viene eseguito in tre ambienti. Cambia il modo in cui i container e i servizi dati vengono avviati, ma non cambiano i contratti API o il contenuto delle immagini. La Tabella 4.1 mette a confronto le tre modalità.

Tabella 4.1: Confronto tra gli ambienti di esecuzione.

Aspetto	Docker Compose	k3d locale	AWS
Orchestrazione	Docker Compose	k3s dentro container Docker	Amazon EKS
Database	container PostGIS	pod PostGIS	Amazon RDS PostgreSQL
Cache	container Redis	pod Redis	Amazon ElastiCache
Messaggistica	container RabbitMQ	pod RabbitMQ	Amazon MQ
Immagini	build locale	build e import in k3d	Amazon ECR
Accesso	localhost:8080	localhost:18080	Load Balancer pubblico
Uso principale	sviluppo e demo rapida	simulazione Kubernetes	dimostrazione cloud reale

4.1 Docker Compose e simulazione k3d

Docker Compose [8] è il percorso più semplice. Il comando seguente costruisce le immagini, crea la rete interna e avvia PostGIS, Redis, RabbitMQ, i servizi e il frontend:

```
1 docker compose up -d --build
```

Il container `db-init` attende PostgreSQL, applica la migrazione e importa i segmenti OSM. I servizi partono solo dopo che le dipendenze richieste risultano healthy. Questa sequenza spiega perché il primo avvio può richiedere più tempo dei successivi. Al termine la GUI è disponibile su <http://localhost:8080>.

La simulazione cloud locale usa k3d [9], uno strumento che esegue k3s dentro container Docker. k3s è una distribuzione leggera e compatibile con Kubernetes [10]; k3d automatizza la creazione del cluster e l'importazione delle immagini locali. Lo script `scripts/cloud_sim_local_up.sh` costruisce i container, crea il cluster, applica i manifest, importa OSM, esegue gli smoke test e mantiene un port-forward verso <http://localhost:18080>.

Su macOS il runtime Docker viene fornito da Colima [11]. Lo script `scripts/colima_start.sh` avvia una sola macchina virtuale Linux ARM64 con 2 CPU, 3 GB di RAM e 20 GB di disco. Questa configurazione lascia memoria sufficiente al sistema operativo e agli strumenti di sviluppo sul computer usato per il progetto, che dispone di 8 GB di RAM.

In una prima ipotesi era stato considerato un ambiente con macchine virtuali Multipass, un cluster Kubernetes completo e il normale runtime Docker Desktop. Questa soluzione avrebbe sommato il costo della virtualizzazione di Docker a quello delle VM e del control plane Kubernetes, occupando una parte eccessiva degli 8 GB disponibili. La combinazione Colima e k3d evita più VM separate: Colima fornisce il motore Docker, mentre i nodi k3s vengono eseguiti come container nello stesso ambiente. Si conserva così una simulazione Kubernetes realistica, ma con tempi di avvio e consumo di memoria più adatti alla macchina disponibile.

PostgreSQL, Redis e RabbitMQ rimangono pod locali, quindi questa modalità non replica i servizi gestiti AWS. Riproduce però gli oggetti Kubernetes più importanti: Deployment, Pod, Service, ConfigMap, Secret, Job, probe di salute e Horizontal Pod Autoscaler. La scelta non sostituisce il test su EKS, ma permette di controllare manifest e interazioni tra pod prima di sostenere i costi del cloud reale.

4.2 Deploy reale su AWS

Il deploy AWS sostituisce i componenti dati locali con servizi gestiti [12]. EKS esegue i pod applicativi; RDS ospita PostgreSQL e l'estensione PostGIS; ElastiCache conserva lo stato Redis; Amazon MQ fornisce RabbitMQ; ECR contiene le immagini Docker. La VPC racchiude rete, subnet e regole di sicurezza, mentre un Load Balancer espone il frontend.

La Figura 4.1 separa i flussi operativi per evitare di confondere traffico utente, pubblicazione delle immagini, configurazione e spegnimento. Lo schema ripete alcuni componenti, come EKS e GitHub Actions, perché ogni riga descrive un processo diverso.

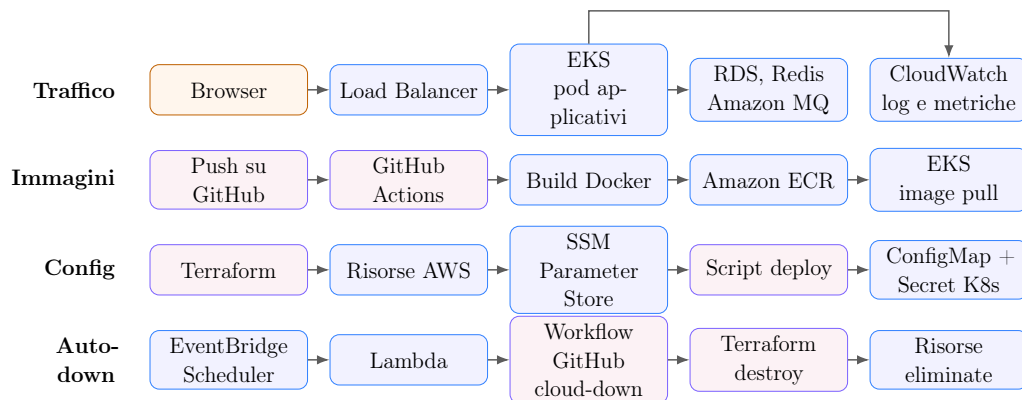


Figura 4.1: Flussi principali del deploy e dell'esecuzione su AWS.

Il comando `scripts/cloud_demo_up.sh` esegue i controlli preliminari, applica Terraform, aggiorna il contesto Kubernetes, prepara ConfigMap e Secret, applica i manifest, importa OSM e attende il completamento dei rollout. Se `PUSH_IMAGES=true`, le immagini locali vengono costruite e caricate in ECR. Con `PUSH_IMAGES=false`, il deploy usa un tag già presente nel registry e risulta più rapido, ma fallisce se l'immagine richiesta non esiste.

Spegnimento automatico. Le risorse gestite hanno un costo continuo, quindi il deploy è pensato per sessioni brevi e richiede conferme esplicite. Dopo quattro ore EventBridge Scheduler invoca Lambda, che legge da SSM un token GitHub e avvia il workflow `cloud-down`. Quest'ultimo esegue la distruzione Terraform: Lambda non elimina direttamente EKS o la VPC.

4.3 Infrastructure as Code, CI/CD, sicurezza e test

Terraform, attraverso il provider AWS [13], descrive l'infrastruttura come codice: VPC, subnet, EKS, node group, RDS, ElastiCache, Amazon MQ, ECR, gruppi di log, ruoli IAM, Lambda e permessi dello scheduler. Lo stato remoto viene conservato in S3, che supporta nativamente il locking nelle versioni recenti di Terraform. In questo modo un account diverso può ricreare l'ambiente dopo aver inizializzato il proprio bucket e adattato backend, credenziali e quote.

Ansible è presente nella repository come esercizio didattico di configurazione. Sarebbe particolarmente utile con macchine virtuali o istanze EC2 gestite direttamente, ad esempio per installare Docker o k3s. Nel percorso principale del progetto svolge un ruolo secondario perché EKS, RDS, ElastiCache e Amazon MQ delegano ad AWS gran parte della configurazione del sistema operativo.

4.3.1 GitHub Actions, ECR e gestione dei segreti

Un push su GitHub avvia i workflow di controllo e build configurati per quel ramo. La CI esegue controlli statici, avvia Docker Compose e lancia gli smoke test. Un workflow separato costruisce in parallelo le immagini dei servizi e, quando nel repository sono presenti credenziali AWS valide, le pubblica in ECR con il tag `latest` e con l'hash del commit. Il tag legato al commit permette di risalire con precisione al codice usato per un'immagine.

Il push non distribuisce automaticamente l'applicazione su EKS. Il deploy rimane un'operazione esplicita, eseguita dagli script locali oppure dal workflow manuale `Deploy EKS`. Questa separazione evita che ogni modifica accenda o alteri risorse a pagamento. Durante un deploy con immagini già aggiornate, EKS può scaricarle direttamente da ECR senza rifare il build sul computer locale.

Le chiavi TomTom, Nemotron ed ElevenLabs non entrano mai nel JavaScript. In locale vengono lette da un file `.env` non versionato. Su AWS sono conservate in SSM Parameter Store come `SecureString`. Gli script di deploy leggono gli endpoint prodotti da Terraform e i parametri SSM, poi generano una `ConfigMap` per la configurazione ordinaria e un `Secret` Kubernetes per password e API key. I pod ricevono variabili ambiente da questi oggetti; non

contattano direttamente SSM durante l'esecuzione.

Le password di RDS e Amazon MQ sono necessarie in cloud, mentre le chiavi dei fornitori esterni restano facoltative. Se mancano, i servizi avviano le rispettive modalità simulate. I log non devono mai stampare valori sensibili. CloudWatch raccoglie i log dell'ambiente AWS con un periodo di conservazione limitato, utile per controllare rollout, errori e invocazioni dell'auto-down senza mantenere dati indefinitamente.

4.3.2 Controlli automatici e verifica operativa

I test sono distribuiti su più livelli. I controlli statici verificano sintassi, contratti OpenAPI, presenza delle integrazioni frontend, configurazione TomTom e coerenza degli script. Lo smoke test usa il gateway come farebbe il browser e controlla salute dei servizi, segmenti, heatmap, sessione e scenari. Il test end-to-end esegue un percorso dimostrativo più completo. La Tabella 4.2 riassume i comandi principali.

Tabella 4.2: Controlli principali disponibili nella repository.

Comando	Verifica
<code>scripts/static_checks.sh</code>	Sintassi, contratti, frontend, scoring, OSM, override e configurazioni.
<code>scripts/smoke_test.sh</code>	Percorsi HTTP principali attraverso il gateway e stato delle dipendenze.
<code>scripts/e2e_demo_test.sh</code>	Flusso dimostrativo con sessione, segmenti, prediction e fallback.
<code>scripts/k8s_smoke_test.sh</code>	Raggiungibilità e comportamento essenziale dopo il deploy Kubernetes.
<code>scripts/load_test.sh</code>	Risposte ripetute e comportamento elementare sotto carico controllato.

Nei manifest Kubernetes sono presenti readiness e liveness probe. Una readiness fallita impedisce al Service di inviare traffico a un pod non pronto; una liveness fallita permette a Kubernetes di riavviarlo. Lo script cloud attende anche il Job di inizializzazione e i rollout prima di restituire l'URL del Load Balancer. Questo evita il caso in cui venga mostrato un indirizzo pubblico mentre database o API non sono ancora utilizzabili.

La validazione attuale controlla soprattutto correttezza funzionale e riproducibilità. Non sostituisce una validazione scientifica della precisione delle stime, perché manca ancora uno storico reale di occupazione con cui confrontare i risultati. Questa differenza viene mantenuta esplicita sia nei test sia nell'interfaccia.

Capitolo 5

Limiti, sviluppi possibili e conclusioni

ParcheggIA dimostra un flusso completo, ma non è ancora un prodotto da usare come unica fonte durante la ricerca di parcheggio. Il limite principale riguarda i dati: TomTom descrive traffico, incidenti e POI, non la disponibilità su strada. Le percentuali restano quindi stime costruite con regole e segnali indiretti. OSM e gli override locali possono inoltre non conoscere una modifica recente alla segnaletica o una limitazione temporanea.

La copertura geografica dettagliata è concentrata su Catania. PMTiles contiene la basemap locale e PostGIS i segmenti importati per la stessa area. Estendere il progetto a un'altra città richiederebbe una nuova estrazione OSM, un archivio cartografico adeguato e fonti affidabili per sosta tariffata e restrizioni.

La navigazione demo rimane volutamente limitata. Il movimento viene agganciato alla rete OSM e il pilota automatico percorre un itinerario preparato, ma non esiste ancora un motore completo che calcoli percorsi, sensi unici e istruzioni. Le API TomTom disponibili includono il routing e potrebbero essere integrate; in alternativa si potrebbero usare OSRM o Valhalla per mantenere il percorso aperto.

Anche il deploy AWS privilegia la dimostrazione rispetto a un ambiente di produzione. EKS, RDS, ElastiCache e Amazon MQ mostrano bene la sostituzione dei componenti locali con servizi gestiti, ma hanno un costo elevato per il traffico previsto dalla demo. Mancano inoltre una gestione multi-ambiente completa, policy IAM più restrittive, un dominio HTTPS stabile e un sistema di monitoraggio con soglie e allarmi operativi.

5.1 Sviluppi possibili

Le evoluzioni più utili, in ordine logico, sono:

1. raccogliere uno storico verificabile di occupazione, report e fasce orarie per misurare l'errore del modello attuale;
2. addestrare un modello predittivo soltanto dopo aver ottenuto dati sufficienti e mantenere il sistema rule-based come baseline di confronto;
3. integrare un motore di routing con sensi unici, istruzioni e ricalcolo del percorso;
4. ampliare e versionare gli override comunali per strisce blu, tariffe e limitazioni;
5. aggiungere dashboard CloudWatch o Prometheus/Grafana con allarmi su errori, latenza e consumo delle API;
6. separare configurazioni dev, demo e produzione e introdurre policy IAM basate sul privilegio minimo;
7. automatizzare test visivi della GUI su schermi desktop e mobile.

5.2 Conclusioni

Il risultato finale mette insieme una parte visibile e una parte infrastrutturale. La prima è una mappa da navigatore con segmenti reali, heatmap, parcheggi, suggerimenti e voce. La seconda segue il dato dal browser ai microservizi, da PostGIS a Redis e RabbitMQ, fino ai provider esterni e ai servizi AWS.

Il passaggio dalle grandi zone ai segmenti stradali ha reso la demo più concreta: un consiglio può indicare una via vicina e la sua percentuale invece di colorare un intero quartiere. L'importazione OSM e gli override hanno fornito una base geografica riproducibile; il modello a regole ha permesso di iniziare senza inventare un addestramento privo di dati; i fallback hanno reso il progetto eseguibile anche dopo un clone pulito.

Dal punto di vista cloud, Docker Compose, k3d ed EKS rappresentano tre livelli dello stesso sistema. Terraform rende ripetibile l'infrastruttura, GitHub Actions controlla il codice e pubblica le immagini, SSM separa i segreti dalla repository e l'auto-spegnimento riduce il rischio economico della demo. L'aspetto più utile del progetto non è quindi una singola tecnologia, ma il modo in cui componenti diversi collaborano mantenendo confini comprensibili e modalità di esecuzione verificabili.

Bibliografia

- [1] MapLibre contributors. Maplibre gl js documentation. <https://maplibre.org/maplibre-gl-js/docs/>.
- [2] Protomaps. Pmtiles documentation. <https://docs.protomaps.com/pmtiles/>.
- [3] OpenStreetMap contributors. Street parking. https://wiki.openstreetmap.org/wiki/Street_parking.
- [4] TomTom. Traffic api documentation. <https://developer.tomtom.com/traffic-api/documentation>.
- [5] TomTom. Search api documentation. <https://developer.tomtom.com/search-api/documentation>.
- [6] NVIDIA. Nvidia nemotron 3 nano 30b a3b. <https://build.nvidia.com/nvidia/nemotron-3-nano-30b-a3b>.
- [7] ElevenLabs. Text to speech api reference. <https://elevenlabs.io/docs/api-reference/text-to-speech>.
- [8] Docker. Docker compose documentation. <https://docs.docker.com/compose/>.
- [9] k3d contributors. k3d documentation. <https://k3d.io/>.
- [10] Cloud Native Computing Foundation. Kubernetes documentation. <https://kubernetes.io/docs/>.
- [11] Colima contributors. Colima: container runtimes on macos and linux. <https://github.com/abiosoft/colima>.
- [12] Amazon Web Services. Aws documentation. <https://docs.aws.amazon.com/>.

- [13] HashiCorp. Terraform aws provider documentation. <https://registry.terraform.io/providers/hashicorp/aws/latest/docs>.